

Joining data from multiple sources

K Arnold

A note on mutate

- Badly named. Think "add_computed_column".
- **DON'T** think of it operating on a variable ("mutate the ride's start time").
- **DO** think of it operating on a data frame ("add a column computed by flooring the start time")

Joining data frames

- I have a data frame **x** (e.g., Covid confirmed cases)
- I want extra information about things in **x** (e.g., population)
- Some other table, **y**, has that information. "Joins" let me look it up.
- Needs a **key**: what has to match. Must match exactly.

x		y	
1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

Graphics thanks to [tidyexplain](#)

Types of joins

What to do when things don't exactly line up? Start with all rows, but:

- **full** or **outer**: Leave blanks (**NA**) for mismatches
- **inner**: Drop rows with any mismatches
- **left** / **right**: Drop rows where one of the sides has a mismatch

Setup

x

```
## # A tibble: 3 x 2
##   key xdata
##   <dbl> <chr>
## 1     1 x1
## 2     2 x2
## 3     3 x3
```

y

```
## # A tibble: 3 x 2
##   key ydata
##   <dbl> <chr>
## 1     1 y1
## 2     2 y2
## 3     4 y4
```

X

1	x1
2	x2
3	x3

y

1	y1
2	y2
4	y4

full_join()

All rows from both **x** and **y**. Leave **NA** for mismatches.

```
full_join(x, y, by = "key")
```

```
## # A tibble: 4 x 3
##   key xdata ydata
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
## 4     4 <NA> y4
```

full_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

inner_join()

All matching rows. Drops mismatches.

```
inner_join(x, y, by = "key")
```

```
## # A tibble: 2 x 3
##   key xdata ydata
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
```

inner_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

left_join()

All rows from **x**.

```
left_join(x, y, by = "key")
```

```
## # A tibble: 3 x 3
##   key xdata ydata
##   <dbl> <chr> <chr>
## 1     1 x1    y1
## 2     2 x2    y2
## 3     3 x3    <NA>
```

left_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

right_join()

All rows from *y*.

```
right_join(x, y)
```

```
## Joining, by = "key"
```

```
## # A tibble: 3 x 3
##   key xdata ydata
##   <dbl> <chr> <chr>
## 1     1  x1    y1
## 2     2  x2    y2
## 3     4 <NA>   y4
```

right_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

Summary

- `full_join()`: all rows from both `x` and `y`
- `inner_join()`: all *matching* rows from `x` where there are matching values in `y`. Multiple matches? Return all combinations.
- `left_join()`: all rows from `x`
- `right_join()`: all rows from `y`

^ are called *mutating joins* (by analogy to the `mutate` verb). Sometimes useful: *filtering joins* and *nest join*:

- `semi_join()`: include a row from `x` only if there's some match in `y`
- `anti_join()`: include a row from `x` only if there's *no* match in `y`
- `nest_join()`: get bundles of all matching rows from `y` (most flexible)

semi_join()

```
semi_join(x, y, by = "key")
```

```
## # A tibble: 2 x 2
##   key xdata
##   <dbl> <chr>
## 1     1 x1
## 2     2 x2
```

semi_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

anti_join()

```
anti_join(x, y, by = "key")
```

```
## # A tibble: 1 x 2  
##   key xdata  
##   <dbl> <chr>  
## 1     3 x3
```

anti_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

We want to keep all rows and columns from `confirmed_cases` and add a column for corresponding populations. Which join function should we use?

```
#fisheries %>% select(country)
```

```
#continents
```

Join fisheries and continents

```
#fisheries <- left_join(fisheries, continents)
```

Join fisheries and continents

```
#fisheries <- left_join(fisheries, continents)
```

How does `left_join()` know to join the two data frames by `country`?

Hint:

- Variables in the original fisheries dataset:
- Variables in the continents dataset:

Check the data

```
#fisheries %>%  
#   filter(is.na(continent))
```

Implement fixes

```
# fisheries <- fisheries %>%  
#   mutate(continent = case_when(  
#     country == "Democratic Republic of the Congo" ~ "Africa",  
#     country == "Hong Kong" ~ "Asia",  
#     country == "Myanmar" ~ "Asia",  
#     TRUE ~ continent  
#   )  
# )
```

...and check again

```
# fisheries %>%  
#   filter(is.na(continent))
```

What does the following code do?

```
fisheries %>%  
  mutate(aquaculture_perc = aquaculture / total)
```

